Agent Commerce Gateway: A Trusted Execution Layer for Delegated Agent Payments

Oct 2, 2026 · @K

Executive summary

Thesis: agents hold authority, not credentials. The PSP validates that authority, evaluates policy and risk, selects an eligible credential, executes the payment, and keeps the evidence needed to resolve disputes. Paze, Visa, Mastercard and other providers supply credentials into that execution layer.

This document answers a broader question than "how does Paze give an agent a secure way to pay". It asks what the trusted transaction architecture for delegated agent commerce is, and what role each of Paze, the networks, the PSP and the agent should play.

Recommendation: JPMMS builds an Agent Commerce Gateway with three parts:

- the gateway itself, which owns agent identity, delegation, payment intents and attempts, evidence and post-purchase events;
- the Decision Platform, which runs delegation policy and the risk layers;
- credential orchestration, which selects an eligible credential binding for each payment attempt.

Paze is the lead issuer-backed credential provider in this design, alongside Visa and Mastercard agentic credentials, Link, and a virtual-card fallback. Building it this way removes the program's dependency on a single EWS decision. It also puts the PSP where agentic commerce needs it, as the owner of trusted transaction execution.

Division of roles

Party	Provides
Consumer	Delegation: the authority, scope and revocation
Agent	Instruction interpretation, cart, merchant interaction, signed requests. Never credentials

Party	Provides
Paze and issuers	Trusted consumer credentials and issuer-grade authentication of the delegation
Visa and Mastercard	Agent-bound tokens, cryptograms, agent registries, and the eventual liability rules
PSP (JPMMS)	Trusted transaction execution: gateway, decisioning, credential orchestration, routing, capture, settlement, disputes, evidence
Merchant	Accepting agent-originated orders, fulfilment, and service

What the consumer sees

The consumer acts twice: once to delegate, then only on exceptions. Everything else is a lightweight activity feed. This experience is what H3 tests: whether consumers will delegate when the limits and the off switch sit in their bank app.

The consumer acts twice: once to delegate, then only on exceptions

1 · Delegate once, in the bank app

Let Instinct buy for you?

Pays with Visa •• 4417. Instinct never sees your card.

Per purchase	**up to $500**
Per month	**up to $2,000**
Where	**US stores: retail, food, travel**
Ask me first	**new store, or over $500**
Ends	**in 6 months**

Allow with passkey

Not now

2 · Normal purchases: no taps, just a feed

Agent activity

This month: $215 of $2,000

Runner's Depot **$142.18**
Running shoes · within limits · 2:14 pm

Green Market **$63.40**
Groceries for two · within limits · Tue

CloudBox **$9.99**
Monthly storage · subscription · Mon

Each line arrives as a quiet notification, not a prompt.

Something wrong? Report a purchase

3 · Exception: one tap, only when needed

Instinct needs your OK
$780.00 at Summit Outfitters
New store, and over your $500 limit

You asked for
"Camping gear for two, under $800"

In the cart
1 tent, 2 sleeping pads

Expires in 5 minutes if you don't answer

Approve with passkey

Decline

4 · Control stays with the bank

Agent permissions

Instinct Active
Visa •• 4417 · $215 of $2,000 used
1 subscription · ends Apr 2027

Pause | Edit limits | **Revoke**

Revoke stops new purchases and subscriptions at once, on every card network.

Also reachable from Paze and from the agent.

consumer experience · 4 moments (illustrative)

Moment	Consumer effort	What happens underneath
1 Delegate	One screen and a passkey, in the bank app (or Paze, or network consent)	Gateway creates the delegation; the credential source provisions a binding
2 Normal purchase	None. A quiet line in the feed after the fact	Policy approves; the attempt executes; evidence is recorded
3 Exception	One tap and a passkey, showing the request next to the cart	Policy returns step-up; the gateway issues the challenge; the attempt resumes or ends
4 Control	Pause, edit or revoke at any time	The delegation state changes; bindings and subscriptions cascade

Experience principles

- **Silent by default:** purchases inside limits never interrupt. The feed informs; it doesn't ask.
- **Interrupt with context:** a step-up always shows what the consumer asked for next to what is in the cart. That is also the moment most likely to catch prompt manipulation.
- **One home for control:** the bank app is the system of control. The agent and Paze can mirror it, but every change lands on the same delegation.
- **Plain limits:** dollars, stores and dates rather than MCC codes. The consumer's settings compile into policy (Section 7).
- **Reporting is easy:** "Report a purchase" from the feed starts the misexecution path (Section 14) with the evidence already attached.

Screens are illustrative. Brand, copy and placement inside each issuer's app are for issuers and Paze to decide.

What changed from v1

v1 mixed three propositions as if they were one solved system: a credential architecture, a delegated-authorization architecture and a merchant-acceptance architecture. v2 separates them, defines the boundaries between them, and moves the PSP from "bridge plumbing" to the center of execution.

Area	v1	v2
Core abstraction	Agent-bound network token	Delegation. The network token is only the credential that executes it
Authorization model	Delegation → cart → payment	Delegation → payment intent → payment attempt, with a separate authorization record
Cart mandate	One object carrying three assertions	Consumer authorization, agent execution and payment authorization modelled separately
Mandate	Static fields	Delegation policy on the Decision Platform
Bridge	Plumbing into Paze acceptance	Agent Commerce Gateway, credential-agnostic and owned by the PSP
Paze	The whole architecture	One credential provider, the lead issuer-backed one
Risk	A short list of controls	Four risk layers plus a threat model that includes prompt manipulation
"Proof of intent"	Signed intent treated as evidence of what the consumer wanted	Formal agent-misexecution path with an evidence chain
Liability	Proposed model	Hypothesis, tested in phases
Merchant impact	"No change"	No new credential integration; new responsibilities for agent-originated orders
Recurring and multi-merchant	One field	Subscription authorization object; a shopping session with many merchant orders
Competitive claims	Paze wins on trust and approvals	Proposed differentiation plus experiments that test it

Area	v1	v2
Metrics	Payment performance only	A funnel for consumers, agents, merchants and payments, plus economics

Kept from v1:

- The agent never sees a PAN or token.
- Credentials are agent-bound, with a cryptogram per cart.
- The consumer delegates, and the bank app can revoke.
- An agent registry, cart hash and replay protection.
- A Paze Agent API, now positioned as a proposal to EWS.

Constraints and hypotheses

Four constraints are facts today. Four of the claims this design depends on are hypotheses, and the roadmap is built to test them before scaling.

Constraints

#	Constraint	Design response
C1	Card-on-file network tokens are bound to one merchant's token requestor	Credentials bind to the delegation and the agent; a cryptogram is minted per payment attempt
C2	Paze has no headless path; checkout assumes the consumer is present	The Paze Agent API is a proposal to EWS. The gateway goes live first on credentials that already have agent paths (C4)
C3	Merchants have no endpoint for agent-originated orders	The gateway exposes ACP, UCP, TAP-verified requests and MCP, and fronts JPMMS merchants first
C4	Agent credentials exist outside Paze (Link one-time cards; Visa and Mastercard agentic tokens)	Credential orchestration supports all of them; Paze is added when its API exists

Two further constraints are about governance and rules, not technology:

- **Governance:** EWS is owned by seven banks. JPM can sponsor a proposal there but can't decide it alone.
- **Network rules:** the networks have not yet published liability or dispute rules for agent transactions.

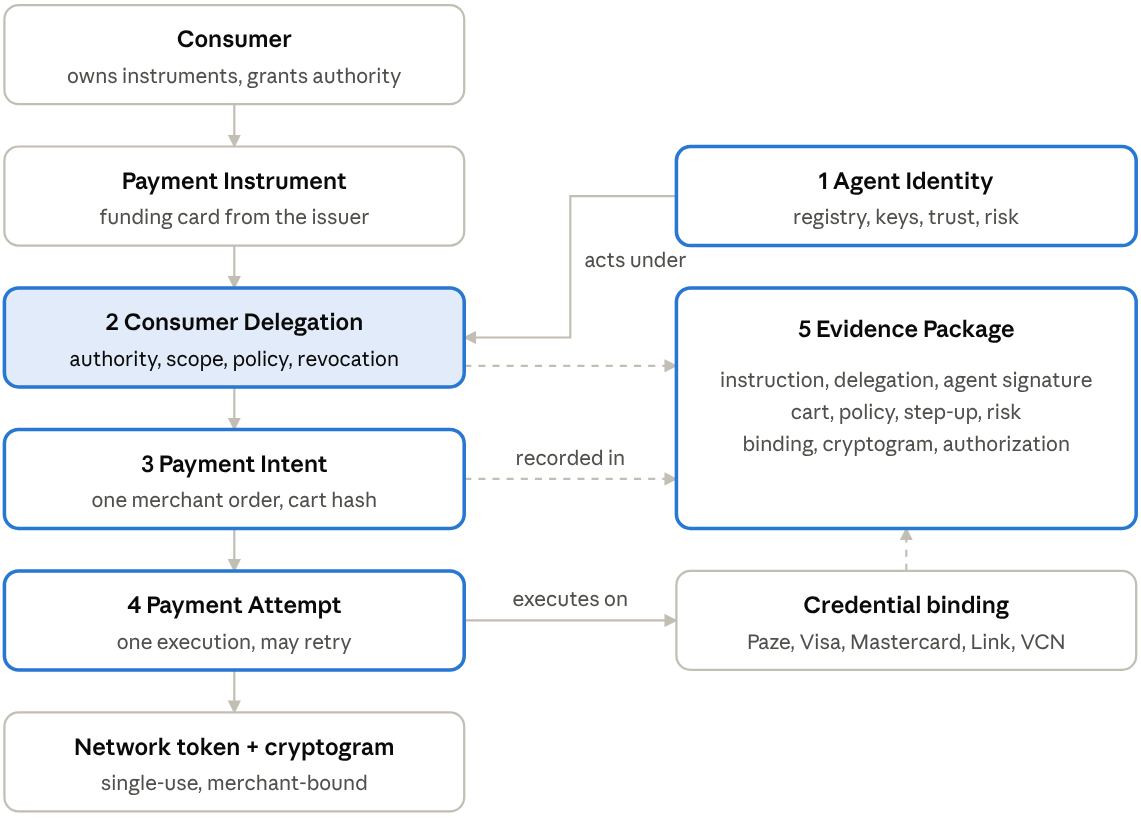
Hypotheses to test, not assume

#	Hypothesis	How it is tested
H1	Issuer-authenticated delegations raise approval rates and lower fraud compared with agent virtual cards	Phase 1 A/B on the same agent and merchants
H2	Bundled evidence (delegation, policy, cart, cryptogram, agent indicator) qualifies for a liability shift	Network sandbox, then bilateral contracts (Section 15)
H3	Consumers will delegate when revocation and limits sit in their bank app	Delegation creation and retention funnel
H4	Merchants will accept agent orders when credential integration is unchanged	Merchant activation in the JPMMS pilot base

Five primitives and the object model

The design rests on five primitives. Authority sits in the delegation; the network token is only the credential that executes an attempt. Keeping these separate lets one consumer hold several cards and one agent hold several delegations, and lets credentials change underneath without touching authority.

Authority lives in the delegation; the token only executes it



Numbered boxes are the five primitives; dashed lines feed the evidence package

object model · 5 primitives

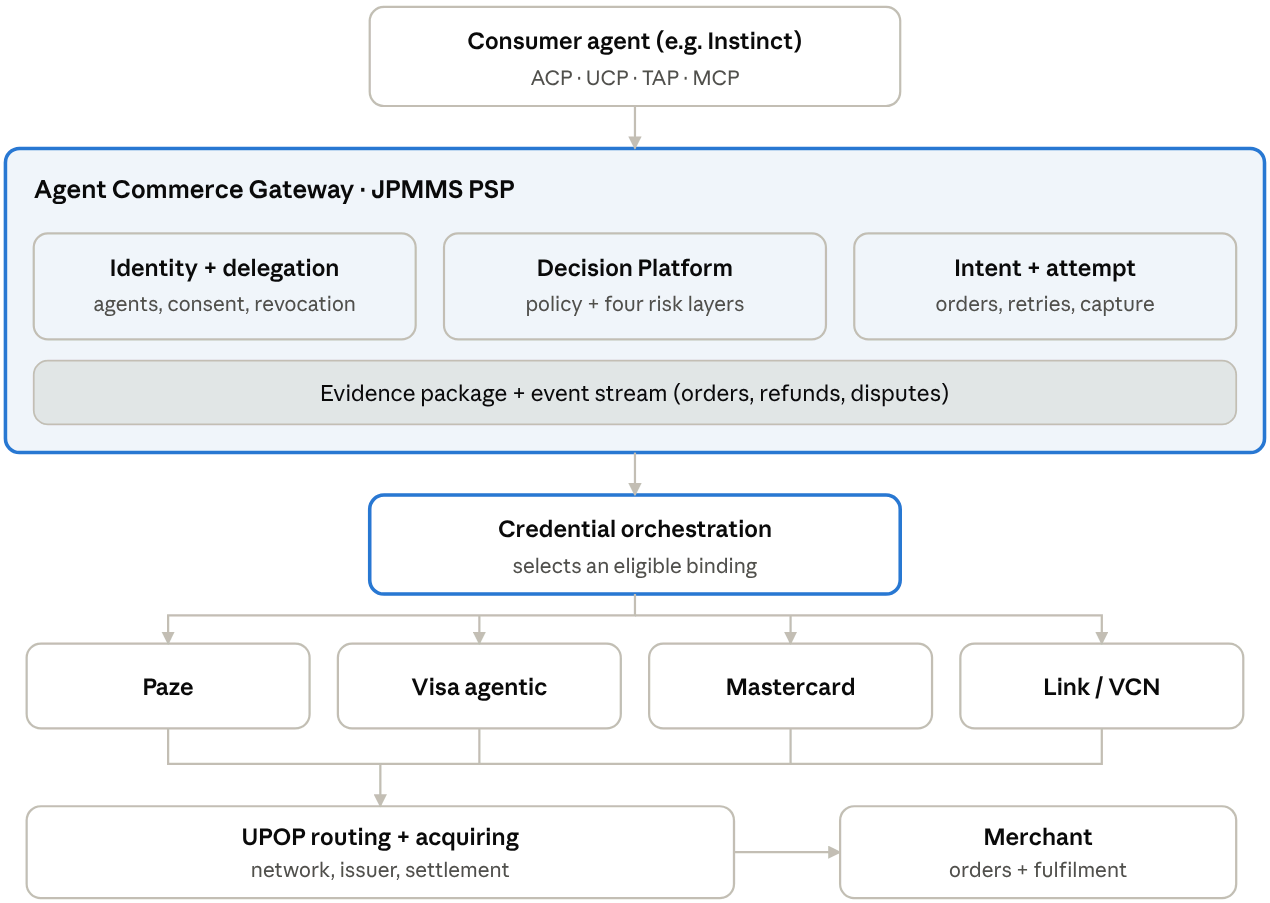
Primitive	Answers	Owner
1 Agent Identity	Who is acting? (agent_id, registration, keys, trust level, status, risk profile)	Network registries; mirrored and scored in the gateway
2 Consumer Delegation	What authority was granted? (limits, scope, validity, step-up, recurring, revocation)	Gateway, authenticated by issuer, Paze or network
3 Payment Intent	What purchase is being paid? (order, merchant, amount, currency, instruction hash, cart hash)	Gateway (PSP resource model)

Primitive	Answers	Owner
4 Payment Attempt	How is it executed this time? (binding, token, cryptogram, route, authorization, retry, capture)	Gateway plus credential orchestration
5 Evidence Package	Why trust or dispute it? (the full chain, Section 14)	Gateway; shared with networks and issuers

Target architecture

Agents talk to one gateway. The gateway decides, then credential orchestration picks a binding, and the payment runs through UPOP routing and acquiring like any other. Paze is one of four credential sources, so the design does not depend on EWS.

The PSP gateway owns execution; credentials plug in beneath it



target architecture · gateway, orchestration, 4 credential sources

What each layer owns:

- **Gateway:** agent authentication, delegation, order creation, intents and attempts, idempotency, agent indicators, evidence and post-purchase events.
- **Decision Platform:** delegation policy, the four risk layers (Section 13) and the step-up decision.
- **Credential orchestration:** selects an eligible binding based on policy, merchant acceptance, network constraints, routing strategy and transaction context (the Decision Platform can later supply an optimization score); token and cryptogram requests, payload translation (for example, Paze encrypted bundles), and fallback to a virtual card.
- **UPOP and acquiring:** routing, authorization, retry, capture, settlement, reconciliation, reporting.

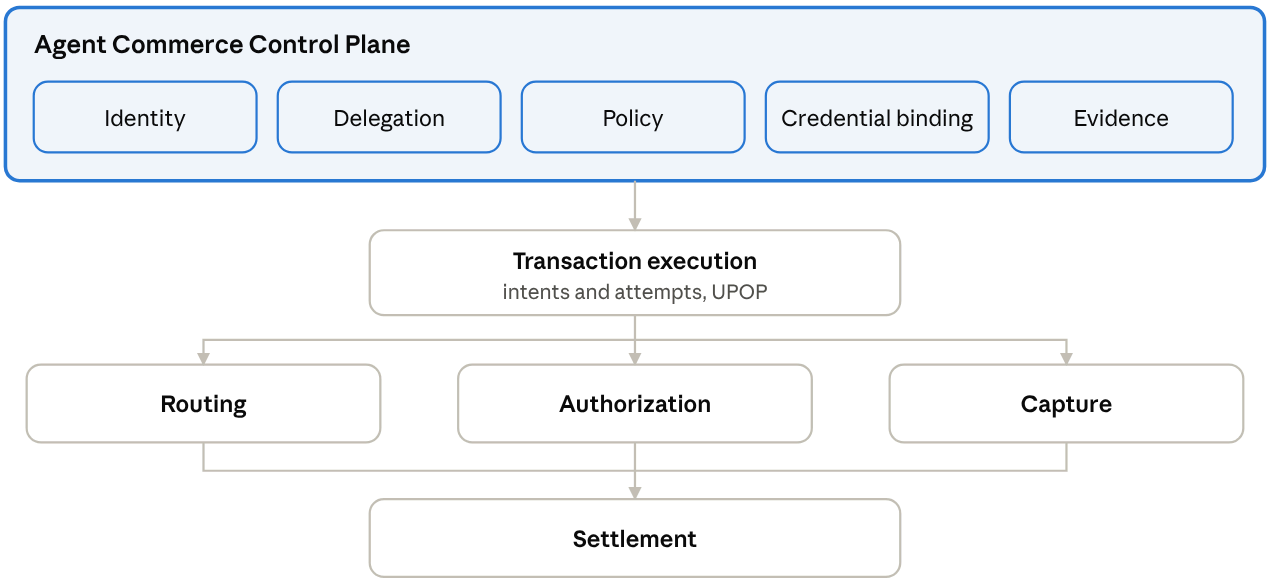
Paze supplies trusted consumer credentials. The PSP supplies trusted transaction execution.

Ownership boundary: the gateway owns state and execution; the Decision Platform owns decisions. The Decision Platform evaluates a delegation but is never its system of record.

Gateway (state and execution)	Decision Platform (decisions)
Agent records, delegation state, orders, intents, attempts, authorizations, evidence, events	Delegation policy, the four risk layers, step-up, agent score, merchant score, the transaction decision, and later the credential-optimization score

Agent Commerce Control Plane. For executive audiences the architecture reduces to two layers. A control plane decides what is allowed, made up of identity, delegation, policy, credential binding and evidence. Execution then makes it happen through routing, authorization, capture and settlement.

The control plane decides what may happen; execution makes it happen



A conceptual layer, not a separate system: its parts live in the gateway and the Decision Platform

control plane · 5 parts over execution

Delegation: three assertions and a policy engine

Three different statements were hidden inside v1's cart mandate. Each is signed by a different party and proves something different. Keeping them apart is what makes disputes resolvable.

Assertion	Statement	Signed by	Proves	Does not prove
Consumer authorization	"This agent may spend up to these limits under these conditions"	Consumer, with passkey at the issuer or Paze	The agent had authority	That any specific purchase was wanted
Agent execution	"I read the instruction as this merchant, these items, this price"	Agent key from the registry	What the agent chose and why	That the choice matched the consumer's intent

Assertion	Statement	Signed by	Proves	Does not prove
Payment authorization	"This exact payment attempt is authorized against this delegation"	Gateway (policy decision) plus network cryptogram	The attempt was inside policy and cryptographically bound	Merchant performance

Policy runs on the Decision Platform. A Paze-specific rules engine is not built. Delegation policy is a new decisioning use case on the existing Decision Platform, so one engine serves Paze, network-direct and Link credentials alike. The consumer's settings compile into a policy, for example:

```
IF   agent = registered AND agent.status = active
AND  merchant_country = US AND mcc IN delegation.allowed_mcc
AND  amount <= delegation.per_txn_cap
AND  period_spend + amount <= delegation.period_cap
AND  merchant_risk < threshold AND agent_risk < threshold
THEN APPROVE
ELSE IF amount > step_up_amount OR new_merchant OR mcc IN high_risk_mcc
THEN STEP_UP
ELSE DECLINE
```

Decision timing. Agent and merchant risk scores are precomputed. Delegation headroom and transaction risk are evaluated in real time on the low-latency decisioning path. The decision is written to the authorization record and to the evidence package.

Delegation object

Field	Notes
delegation_id, consumer_ref, agent_id	One consumer can hold many delegations; one agent serves many consumers
funding_instrument_ref	The Payment Instrument, not a token (Section 8)
limits	Per transaction, per period, per merchant
scope	Merchant list or any; MCC allow and deny; country
validity	Start, expiry, and pause or resume

Field	Notes
step_up_policy	Thresholds, new-merchant rule, category rule
recurring_policy	Whether the agent may create subscription authorizations, and under what caps (Section 9)
revocation	Channels (bank app, Paze, agent) and cascade rules

Credential binding beneath the delegation

Design principle: the delegation is stable and the credential binding underneath it can change. A card reissue, a network switch or a fallback to a virtual card changes the binding, never the consumer's grant of authority.

v1 fixed the cardinality at one token per card, agent and delegation. v2 does not hard-code that. Each delegation holds one or more **credential bindings**, and credential orchestration chooses one per payment attempt.

Credential source	Binding	When orchestration picks it
Paze (agentic)	Agent-bound network token provisioned by Paze; Paze-encrypted payload per attempt	Delegation funded by a Paze card; merchant accepts Paze; EWS Agent API live
Visa agentic credential	VTS agent-scoped token through Visa Intelligent Commerce	Visa card, enrolled through the issuer or the network directly
Mastercard Agent Pay	MDES Agentic Token bound to agent identity and mandate controls (Eco)	Mastercard card
Link and other agent wallets	One-time card per purchase	The agent arrives already bound to that wallet
Fallback virtual card	Single-use card scoped to amount and merchant	The merchant or rail doesn't support agentic tokens

Binding lifecycle

- **Reissue or expiry:** the network updates the token and the binding refreshes. The delegation is untouched.
- **Consumer changes funding card:** a new binding is created and the old one retired. The delegation_id is unchanged.
- **Network or route constraints:** a binding can be preferred per network, issuer, debit or credit, or region. Debit bindings must keep an unaffiliated network option for Reg II routing.
- **Revocation:** every binding is suspended at its source when the delegation is revoked.

The agent never holds a binding, only the delegation handle. Token, cryptogram and payload live with Paze, the networks and the PSP vault (Credential Vault).

Payment intent, attempt, multi-merchant and recurring

"The consumer authorized the agent" and "this payment attempt is authorized" are different facts, so they are different objects. The model reuses the PSP's canonical Order → Payment → Session → Attempt resource model and adds agent context.

The hierarchy

```
Consumer Instruction (intent_hash)
  └─ Agent Shopping Session
       ├─ Merchant Order A
       │    └─ Payment Intent
       │         ├─ Payment Attempt 1 ── Agent Payment Authorization
       │         └─ Payment Attempt 2 ── Agent Payment Authorization
(retry, new route or binding)
       ├─ Merchant Order B
       │    └─ Payment Intent ── Payment Attempt ── Agent Payment
Authorization
       └─ Merchant Order C
            └─ Payment Intent ── Payment Attempt ── Agent Payment
Authorization
```

One instruction such as "buy everything I need for dinner" can fan out to several merchants. Each merchant order gets its own intent and attempts, and all of them share one parent session and one instruction hash.

Agent Payment Authorization belongs to the attempt. It is the record, created when policy approves that attempt, which proves the attempt was permitted. A retry, a route change or a credential change creates a new attempt with its own authorization.

Field	Purpose
authorization_id, payment_attempt_id, payment_intent_id	Links to the resource model
delegation_id, agent_id, session_id	Authority and context
consumer_instruction_hash, cart_hash	Binds what was asked to what was bought
merchant_id, order_id, amount, currency	Execution binding
funding_instrument_ref, credential_binding_ref, network, cryptogram_ref, enforcement_level	Which credential executed it, and whether its limits are network-enforced or only advisory (for example, a virtual-card fallback)
policy_decision, risk_decision, step_up_result	Why it was allowed
expires_at, single_use, capture_mode	Lifecycle control

Lifecycle cases the intent/attempt split handles:

- **Retries:** a new attempt on the same intent, possibly on another binding or route. No new consumer action.
- **Delayed and partial capture, split shipments:** several captures against one authorization, up to the authorized amount.
- **Incremental authorization** (lodging, car rental, tips): increments are checked against delegation headroom, with step-up above a threshold.
- **Cancellation and refunds:** tied to the intent, flowing back to the delegation's spend counters.
- **Marketplaces and multi-pay-in orders:** several attempts or sub-merchants under one intent.

Subscription Authorization is a child of the delegation for recurring and merchant-initiated transactions. Without it, the design only works for one-time purchases.

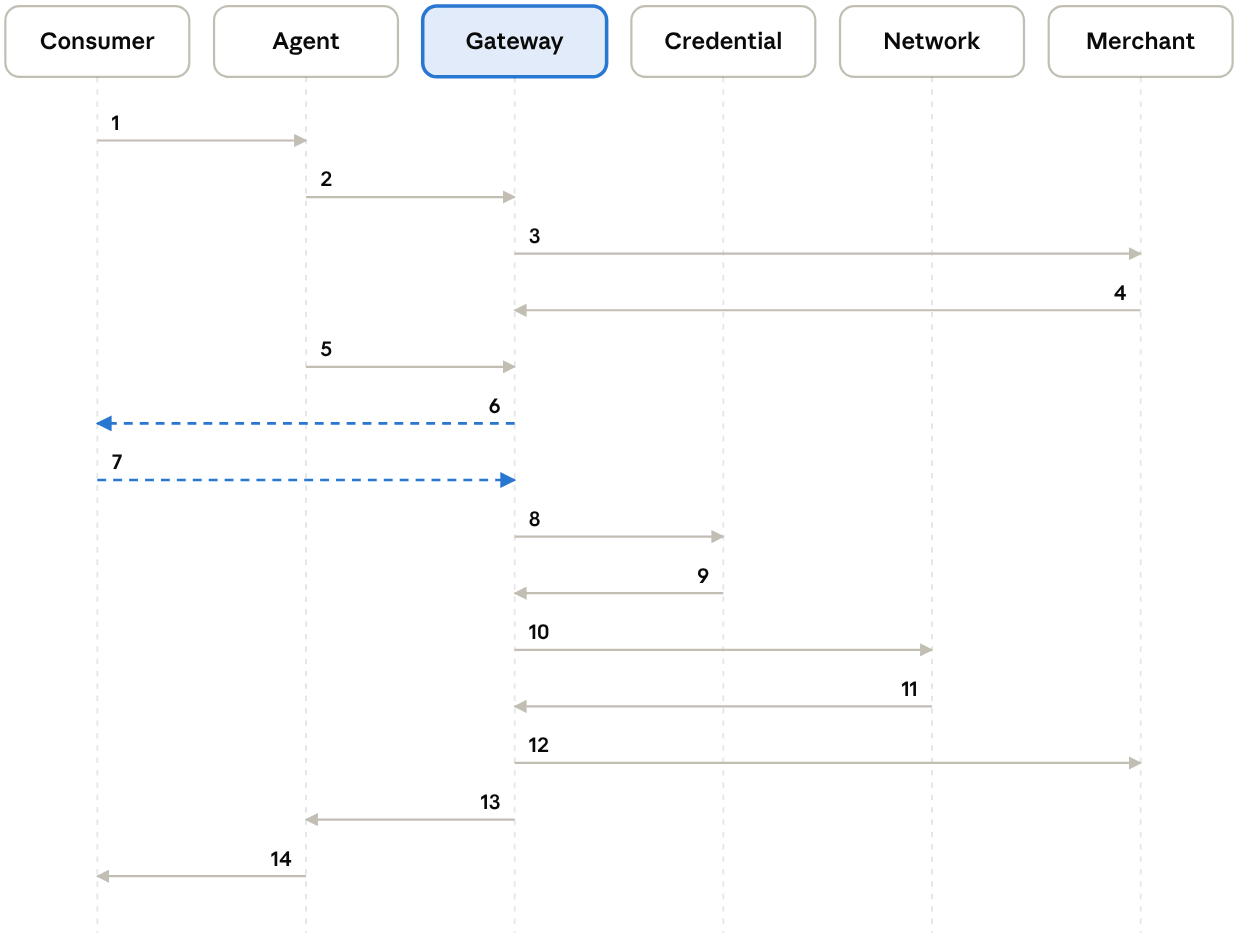
Field	Notes
merchant_id, product or service	Bound to one merchant and one offering
initial_amount, max_recurring_amount	Variable-amount rules, such as utilities or replenishment

Field	Notes
frequency, duration, trial-to-paid rule	Trial conversion requires step-up by default
change rules	A price rise above the cap, or a merchant change, needs consumer step-up
cancellation	Consumer, agent or bank app; propagates to the MIT credential
MIT linkage	The network transaction ID from the first customer-initiated transaction, for later MITs

Purchase flow through the gateway

Every step passes through the gateway, and the agent only ever handles orders, signed assertions and outcomes. A purchase inside policy needs no consumer action.

The gateway sits in every step; the agent never touches a credential



Dashed accent: step-up, only when policy says so. Network includes the issuer.

purchase flow · 14 messages, 1 conditional step-up

1. Consumer gives the agent an instruction.
2. Agent opens a session and submits a signed agent-execution assertion (merchant, cart, price).
3. Gateway creates the order at the merchant and confirms price and stock.
4. Merchant confirms; the gateway creates the payment intent.
5. Agent confirms the payment intent.
6. *If policy says step-up:* the gateway sends a challenge to the bank app, Paze or the network.
7. *If policy says step-up:* the consumer approves with a passkey.
8. Credential orchestration picks a binding and asks the source (for example Paze) for a payload or cryptogram.
9. The source returns a single-use, merchant-bound payload.

10. The gateway authorizes through UPOP, carrying agent indicators and risk signals.
11. The network and issuer approve. The gateway writes the authorization record and the evidence package.
12. Gateway confirms payment to the merchant.
13. Gateway confirms the order and payment to the agent.
14. Agent sends the receipt to the consumer.

Enrollment happens once per agent and funding card. The agent calls POST /delegations; the consumer is redirected to the consent screen of the credential source; the source provisions the binding; the gateway stores the delegation and returns the handle.

Revocation works from the bank app, Paze, the agent or the gateway. It cascades to every binding and subscription, and from the next attempt the gateway refuses.

API surfaces

There are two API surfaces. The Gateway API is JPMMS's to build now. The Paze Agent API is a proposal to EWS, plugged into the gateway's credential orchestration. Names and fields are proposals.

Gateway API (JPMMS, agent-facing; exposed natively and through ACP, UCP and MCP)

Endpoint	Purpose	Returns
POST /delegations	Start a delegation; redirects to issuer, Paze or network consent depending on the credential	delegation_id, bindings available, scope
POST /sessions	Open an agent shopping session for one consumer instruction (instruction hash)	session_id
POST /sessions/{id}/orders	Create a merchant order with a signed agent-execution assertion (cart, merchant, price)	order_id, payment_intent_id

Endpoint	Purpose	Returns
POST /payment-intents/{id}/confirm	Ask for payment; runs policy and risk, picks a binding, mints the credential	authorized / step_up (challenge_id) / declined
GET /challenges/{id}	Step-up outcome (or webhook)	approved / rejected / expired
POST /subscriptions	Create a subscription authorization under a delegation	subscription_id
DELETE /delegations/{id}	Revoke; cascades to bindings and subscriptions	Revoked
Webhooks	Order, payment, capture, refund, dispute, lifecycle and revocation events	Event stream

Idempotency keys are required on every write. Retries create attempts, never duplicate intents.

Paze Agent API (proposal to EWS; called by the gateway, not by agents)

> **Illustrative API: a proposed interface, not a Paze or EWS specification.** Nothing below has been agreed with EWS. Endpoint names, provisioning and payload-encryption semantics are placeholders for discussion.

Endpoint	Purpose
POST /agent/delegations	Paze-hosted consent; Paze provisions the agent-bound token
POST /agent/payloads	Gateway submits the authorization record; Paze returns a payload encrypted to the merchant's Paze key, plus its own risk signals
POST /agent/step-up	Paze or issuer-app challenge, when the gateway's policy or Paze's own policy requires it
DELETE /agent/delegations/{id}	Revoke at Paze and at the network

Endpoint	Purpose
Events	Card lifecycle and consumer-initiated revocation, back to the gateway

Routing Paze calls through the gateway, rather than straight from agents, means EWS integrates once with PSPs instead of with every agent. The payload still decrypts on the existing Paze path (J.P. Morgan Paze docs).

Security baseline

- Mutual TLS and HTTP message signatures, using the agent key registered with Visa TAP and Mastercard's registry.
- A nonce and short TTL on every signed assertion.
- Single-use payloads bound to merchant, amount and currency.
- No credential material ever returned to an agent.

Merchant impact

What can be defended is "no new credential integration". "No merchant changes" can't be: accepting agent-originated commerce is a new merchant responsibility.

Area	Change for a JPMMS merchant	Who carries most of it
Credential decryption and authorization	None: the existing Paze path or existing network-token path	PSP
Agent order intake	Accept orders created through the gateway (catalog, price and stock confirmation)	Merchant, with gateway tooling
Agent identity and terms	Recognize agent-originated orders; decide whether agent terms apply	Merchant policy, with gateway signals
Fraud policy	Rules aware of agent indicators and agent risk	PSP decisioning, merchant-tunable
Returns, refunds, cancellation	Route events back to the agent and consumer	Gateway event stream

Area	Change for a JPMMS merchant	Who carries most of it
Customer communication	Receipts and notices reach both the consumer and the agent	Merchant, with gateway fan-out
Disputes	Respond using the evidence package	PSP assembles; merchant reviews

The PSP's job is to absorb as much of this as possible, so that "agent-ready" becomes a configuration for JPMMS merchants rather than a project.

Risk architecture and threat model

Agent commerce needs four risk layers, not a list of controls. Its most distinctive threat is an authenticated agent that has been manipulated, which is different from stolen credentials.

Four risk layers (each feeds the Decision Platform; precomputed where possible, real-time where needed)

Layer	Signals	Timing
Consumer	Card and account history, issuer risk, device and authentication strength at delegation	Precomputed; refreshed on events
Agent	Registry status, reputation, dispute and misexecution rate, key age, incident flags	Precomputed per agent
Delegation	Age, scope breadth, velocity, headroom, step-up history, recent edits	Real-time counters
Transaction	Merchant risk, cart composition, price against reference, new merchant, MCC, amount anomaly	Real-time

Threat model

Threat	Defense
Compromised agent platform	Agent-level kill switch; network registry suspension; per-agent velocity caps

Threat	Defense
Stolen delegation handle	Handle is useless without the agent's signing key; device-bound agent keys; revocation
Agent impersonation	Registry plus signed requests (TAP, HTTP message signatures)
Replay	Nonce and TTL on every assertion; single-use payloads
Cart modification after approval	Cart hash bound into the authorization record
Merchant substitution	Merchant ID bound into the cryptogram and the payload encryption
Amount manipulation	Amount bound into the cryptogram; capture can't exceed the authorization
Credential theft from the agent	The agent never receives a credential
Credential substitution	Bindings resolved server-side from the delegation only
Prompt or instruction manipulation (injected content, a malicious merchant page, a poisoned tool)	Instruction → cart evidence; anomaly checks on price, merchant and category against the instruction; step-up when the cart diverges from the instruction category
Tool or plugin compromise inside the agent	Signed agent-execution assertion; anomaly checks per agent version
Excessive spending	Headroom and velocity policy
Merchant fraud	Merchant risk scoring; agent-acceptance onboarding
Network credential abuse	Network token domain controls and cryptogram binding

Compliance

- PCI DSS: the agent stays out of scope.
- Reg E and Reg Z: dispute paths, see Section 14.

- UDAAP: plain consent screens and easy revocation.
- Privacy: agents receive delegation status and outcomes only.
- Reg II: debit routing choice preserved through the bindings.

Agent misexecution and the evidence package

A signed intent proves the agent had authority. It does not prove the consumer wanted this merchant, this SKU, this warranty or this subscription. So disputes are resolved by finding where along the chain the purchase diverged from the instruction, not by presuming the consumer bears the loss.

For example: the consumer asks for "the best laptop under $1,500", and the agent buys a $1,499 gaming laptop with an add-on warranty. The authority is valid. Whether the purchase matched the intent is a separate question.

Evidence chain. At each link, the record shows who asserted what:

1. Consumer instruction (hash; text retained by the agent under consent)
2. Delegation scope and policy in force
3. Agent interpretation: signed agent-execution assertion, with the agent version
4. Merchant offer: price, terms and add-ons as presented to the agent
5. Cart as submitted (cart hash)
6. Policy and risk decision, and any step-up result
7. Payment authorization: credential binding, cryptogram reference, authorization response
8. Fulfilment and post-purchase events

Divergence decides responsibility

Where it diverged	Indicative responsibility	Status
Credential or delegation compromised	Issuer / credential provider (fraud)	Needs a network rule
Approved outside policy	Gateway / policy owner	Contractual in pilot
Agent interpretation departed from the instruction	Agent platform	Needs a network "agent misexecution" reason code; contractual in pilot

Where it diverged	Indicative responsibility	Status
Merchant offer differed from what was charged or delivered	Merchant	Existing reason codes
Purchase matched the instruction and the consumer changed their mind	Consumer (merchant returns policy)	Existing practice

Agent Transaction Evidence Service. Evidence is a named platform capability, not a gateway feature. The PSP becomes the system of record for agent-transaction evidence, assembling the package automatically for every agent transaction. One service then serves:

- disputes and representment
- fraud investigation
- network liability programs (Section 15)
- merchant support and consumer disputes
- regulatory evidence
- agent reputation, which feeds the agent risk layer

Exposed to networks and issuers in a standard format, it is among the most valuable assets the PSP can own in agent commerce.

Liability as a phased hypothesis

Liability is the biggest business blocker, and it is a hypothesis to test, not a model to assume. The networks have not set liability rules for agent transactions.

Candidate liability-shift bundle. The test is whether this combination earns fraud liability on the issuer side, as 3DS does:

- a consumer-authenticated delegation (passkey at the issuer, Paze or the network)
- an authenticated agent (registry plus signed requests)
- a policy decision recorded against the delegation
- cart integrity verified (cart hash)
- a cryptogram bound to merchant and amount
- an agent indicator received by the merchant and in authorization

Phases

Phase	Liability basis	What it proves
Pilot	Bilateral contracts between JPMMS, pilot merchants, the agent and JPM as issuer	Fraud and dispute rates when the bundle is present
Network launch	A network-defined agentic framework, informed by pilot data	That the bundle maps to a liability shift and to reason codes
Scale	Standardized network rules across Visa, Mastercard and Paze	Merchant economics at scale

Proposed differentiation and experiments

This section describes differentiation; it doesn't claim wins. Each claim is paired with the experiment that would show it. Instinct today pays through Link one-time cards and Shop Pay (Fintech Brainfood; eesel), so Link is the baseline.

Paze's proposed differentiation as a credential provider

1. Issuer-authenticated delegation
2. Broad issuer distribution: cards preloaded and nearing 200 million (Digital Transactions)
3. Network-token lifecycle
4. Mandate enforcement backed by the issuer
5. Network-token security
6. Potential liability treatment (H2)
7. Existing Paze merchant acceptance

The gateway's proposed differentiation as a PSP

1. One integration for agents across every credential
2. One integration for merchants across every agent protocol
3. Decisioning, evidence and disputes built in
4. Agent-aware routing and retries

Experiments

Experiment	Compares	Primary metric
E1 Approval	Issuer-authenticated agentic token vs Link one-time card, same agent and merchants	Authorization rate
E2 Fraud	Same split as E1	Fraud bps, dispute rate
E3 Delegation	Bank-app consent vs agent-hosted consent	Delegation creation and 90-day retention
E4 Step-up	Policy thresholds A vs B	Step-up rate, completion, abandonment
E5 Merchant	Gateway-configured vs bespoke agent integration	Time to agent-ready; agent order share
E6 Evidence	Representment with vs without the evidence package	Dispute win rate

Competitive landscape: agent wallets (Crossmint)

Agent wallets such as Crossmint are fast, PSP-less competitors on the agent side. They are also the most likely early source of agent traffic at JPMMS merchants, today unlabelled. The gateway should treat them as a partner class to convert, not only as a threat. Analysis is based on Crossmint's open-source sample app and its API contract (repo; API.md).

How Crossmint works

1. **Vault:** the consumer enters a card into a white-label Crossmint component inside the agent's app, stored in Crossmint's PCI vault.
2. **Budget approval per purchase:** the agent requests an amount, a description, an optional merchant and an expiry. The user approves with a passkey on a page the agent app hosts (15 minutes by default), creating an "order intent".
3. **Rails in fixed order:** first a Visa Intelligent Commerce or Mastercard Agent Pay agentic token, with the limit enforced by the network. Fallback is an encrypted card, which the server decrypts to a full card number and CVC; that limit is advisory. Every response carries an `enforced` flag, and credentials are minted per merchant.

4. **Checkout by browser automation:** "Agent Checkouts" drives the store's real checkout in a browser, using UCP where available. A maximum cost is enforced, the payment step is answered with the order intent, and store logins are stored as protected inputs in persistent browser profiles.
5. **Distribution:** in-app components, approval links in messaging apps, an MCP server, and a CLI with a skill for coding agents.

Against the gateway

Dimension	Crossmint	Agent Commerce Gateway
Consent	Each budget approved individually	Standing delegation plus policy; silent inside limits
Authentication of consent	The agent app's passkey, plus network verification where a rail needs it	Issuer, Paze or network
Merchant reach	Any website, through browser automation	Merchants with agent endpoints through the gateway
Merchant awareness	Low; looks like an ordinary online card payment	Explicit agent indicators and order intake
Credential	Agentic token, falling back to a full card number	Agentic token or Paze payload; virtual card as last resort
Recurring and incremental	Not expressible beyond one budget at a time	Subscription authorization and incremental policy
Evidence and disputes	Merchant absorbs chargebacks; no evidence chain	Evidence Service and misexecution path
PSP role	Bypassed	The execution layer

Where they are strong: time to market with no merchant, PSP or EWS dependency; a clean approval UX (a purpose string, link approvals for agents with no UI, protected inputs); and honesty about enforcement through the `enforced` flag.

Where they are weak:

- The fallback rail hands a full card number to a server under an advisory limit, which is card sharing with extra steps.

- Browser automation is exposed to bot detection, merchant terms of service and checkout changes.
- Persistent store sessions concentrate breach risk.
- Approving each budget doesn't scale to subscriptions or replenishment.
- Dispute liability lands on merchants who never opted in.

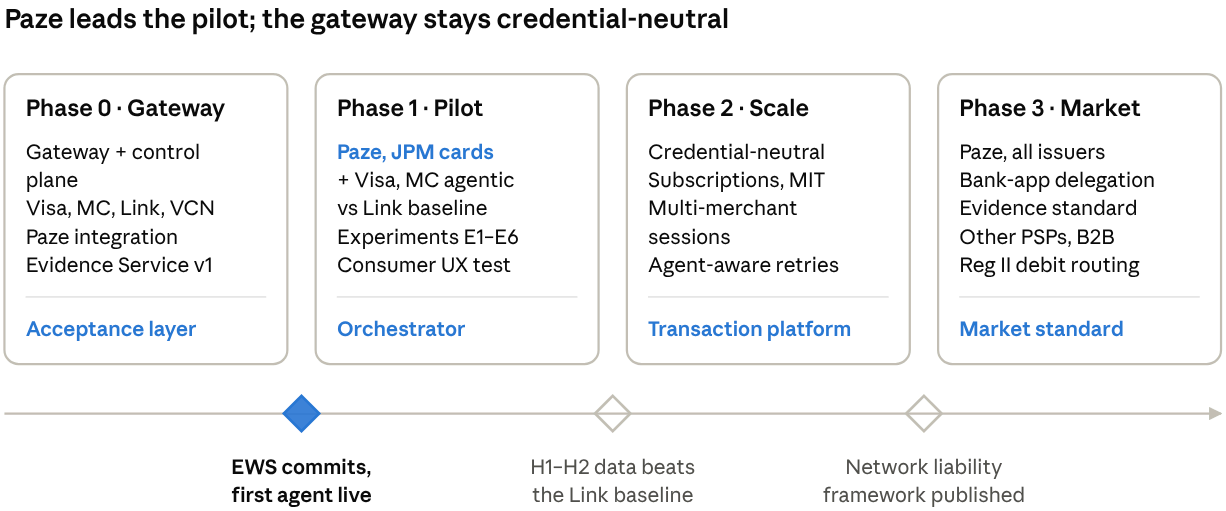
Implications and actions

#	Implication	Action	Phase
A1	Agent traffic from wallets already reaches JPMMS merchants as ordinary online card payments from headless browsers	Detect agentic tokens and agent indicators in UPOP, and tune fraud rules so legitimate agent traffic isn't declined as bot traffic	0
A2	Crossmint prefers UCP when a merchant exposes it	Expose UCP and ACP for JPMMS merchants through the gateway, so wallet traffic moves from browser automation to the sanctioned path, with agent signals and evidence	0
A3	Agent wallets are a credential source in their own right	Add an "agent wallet" credential-source class to credential orchestration, with the enforcement level recorded on every authorization	1
A4	Card entry and periodic CVC re-entry are friction for Crossmint	Position Paze's preloaded cards as the onboarding advantage, and explore Paze as a card source behind third-party agent wallets	1–3
A5	Their approval UX patterns are good	Adopt the purpose string, link approvals for agents with no UI, and idempotent message IDs in the gateway API	0

Bottom line. Even a fast-moving agent wallet ends up rebuilding order intents, per-merchant credentials, rail ordering, cost ceilings and protected inputs. Those are execution-layer concerns. The PSP is where they belong at scale, with the merchant aware and the evidence kept.

Phased roadmap: the PSP's role grows

Paze is the pilot's lead credential in Phase 1, on a gateway that stays credential-neutral. JPMMS builds the gateway, proves agentic execution with Paze alongside network-direct credentials, scales the platform, then expands Paze to every participating issuer.



phased roadmap · 4 phases, 3 gates, PSP role per phase (not to scale)

The accent line in each phase is the role the PSP plays in it. Timing is set at the first gate; no dates are committed before then.

Phase	Goal	PSP role	What JPMMS builds	Credentials	Exit gate
0 · Gateway	Build the Agent Commerce Gateway	**Acceptance layer**: JPMMS merchants become agent-ready	Gateway core and control plane (identity, delegation, intent and attempt on the resource model); delegation policy on the Decision Platform; agent indicators in UPOP; Evidence Service v1; ACP, UCP and MCP endpoints; Paze integration built against the proposed Agent API	Visa agentic, Mastercard Agent Pay, Link, virtual card	EWS commits to a Paze Agent API; first agent live; pilot contracts signed
1 · Pilot	Prove agentic execution with Paze	**Credential orchestrator**: selects an eligible binding per attempt	Paze binding on JPM-issued cards, with Paze delegation in the bank app; credential orchestration with A/B routing; misexecution dispute path; consumer-experience test for H3; experiments E1–E6	**Paze (lead)**, plus network-direct agentic tokens, measured against the Link baseline	H1 and H2 data at or better than the Link baseline

Phase	Goal	PSP role	What JPMMS builds	Credentials	Exit gate
2 · Scale	Scale a credential-neutral agent transaction platform	**Transaction platform**: owns the full lifecycle	Subscription authorization and MIT; multi-merchant sessions; agent-aware retries and routing; merchant self-serve agent readiness; unit-economics reporting	Paze on JPM cards, all network-direct credentials, Link, virtual card	Network liability framework published
3 · Market	Expand Paze; become the market standard	**Market standard**: others build on the JPMMS gateway	Paze at full issuer breadth with delegation in every participating bank app; Evidence Service as a network-recognized format; gateway access for other merchants and PSPs; agentic B2B; Reg II-compliant debit agent routing	Every agentic credential, Paze across all participating issuers	—

Low-regret reuse. Phase 0 is built mostly from existing assets:

- UPOP for routing and authorization;
- the Decision Platform for policy and risk;
- Credential Vault for bindings;
- the Order → Payment → Session → Attempt resource model.

The genuinely new components are delegation, agent identity, credential orchestration for agentic sources, and the evidence package.

The dependency is explicit: the Phase 1 pilot needs EWS to commit to the Paze Agent API at the first gate. Phase 0 builds the Paze integration against the proposed interface so the pilot can start on day one. If EWS isn't ready, Phase 1 begins on network-direct

credentials and Paze joins the pilot as soon as its API lands. Nothing in the architecture changes either way.

Metrics and economics

Success is measured as four funnels plus a unit-economics model. Targets are set from the Phase 1 baseline; no figure here is a commitment.

Funnel	Stages
Consumer	Eligible consumers → delegations created → delegations retained (30/90 days) → first purchase → repeat purchase
Agent	Agents integrated → active agents → transactions per agent → successful transactions
Merchant	Merchants reachable → agent-ready merchants → agent orders created → payments accepted
Payment	Authorization rate · fraud bps · step-up rate and completion · decline rate · retry recovery · dispute rate and win rate · misexecution rate

Unit economics per agent transaction. This must be modelled before Phase 2 funding.

Line	Side	Notes
TPV and take rate	Revenue	Gateway fee, PSP processing, value-added decisioning and evidence
Incremental merchant revenue	Value	Agent-channel orders that would otherwise go to another PSP
Network fees	Cost	Agentic token and cryptogram fees, if any
Paze fees	Cost	Not yet defined by EWS
Gateway run cost	Cost	Decisioning, vault, events, evidence storage
Fraud and dispute losses	Cost	Split by the liability phase

Line	Side	Notes
Issuer economics	Partner	Interchange and fraud, for JPM as issuer and other issuers
Agent economics	Partner	What agents pay or earn; this decides adoption

Objections and open questions

Objection	Answer
"Isn't this just a Paze project waiting on EWS?"	No. The gateway goes live on credentials that already have agent paths. Paze plugs in when its API exists.
"Stripe already does this for agents."	Stripe's stack serves Stripe merchants. The gateway serves JPMMS merchants and is credential-neutral, including Link as one source.
"Networks will go direct to agents."	Network programs provide credentials and identity, not order orchestration, capture, settlement or disputes. The PSP still executes.
"Merchants won't change anything."	Credential integration is unchanged. The new order-intake responsibilities are absorbed by the gateway as configuration (Section 12).
"Liability is unknown."	Agreed. Pilot under contract, collect the data, then take the bundle to the networks (Section 15).
"Agents can be manipulated."	Yes. That is why the evidence chain and the instruction-divergence checks are core features rather than add-ons (Sections 13–14).
"Building a gateway is expensive."	Most components reuse UPOP, the Decision Platform, Credential Vault and the resource model. The genuinely new parts are delegation, agent identity and evidence.

Open questions

- [ ] Which agent platform launches the pilot, and does it accept gateway-hosted delegation?
- [ ] Will EWS commit to a Paze Agent API that is called by PSPs rather than by agents?
- [ ] Will Visa and Mastercard sandbox the liability bundle, and which reason codes cover agent misexecution?
- [ ] Where does instruction text live, and for how long? There is a tension between privacy and evidence.
- [ ] How is Reg II dual-network routing preserved for agentic tokens on debit cards?
- [ ] Pricing: is the gateway fee charged to the merchant, the agent or both?
- [ ] Resolved: the gateway owns delegation and evidence state; the Decision Platform owns decisions (Section 6).

Sources

- J.P. Morgan Developer: Paze payment method
- Digital Transactions: Paze eyes more expansion
- Fintech Brainfood: Instinct and Link
- eesel: Instinct AI explained
- Eco: Mastercard Agent Pay vs Visa Trusted Agent
- The Paypers: Visa Intelligent Commerce Connect
- ACI Worldwide: Paze enablement